

C# / .NET Learning Roadmap

Topic Notes — Section 12: Modern .NET — Application Hosting and Configuration

Topics: 12.1 through 12.7

Date: 29 June 2026

Section 12 — Modern .NET: Application Hosting and Configuration

This section covers all seven roadmap topics: 12.1 (.NET CLI and project/solution structure), 12.2 (DI container — IServiceCollection), 12.3 (DI service lifetimes), 12.4 (Configuration system), 12.5 (Logging — ILogger), 12.6 (Hosting model — Generic Host, WebApplicationBuilder), 12.7 (Environments).

Teaching order: 12.1 (CLI/structure) -> 12.6 (hosting model) -> 12.2+12.3 (DI) -> 12.4 (configuration) -> 12.5 (logging) -> 12.7 (environments).

12.1. .NET CLI and Project/Solution Structure

Builds on: 1.1 (.NET ecosystem overview), 1.2 (environment setup, project/solution structure).

What is the .NET CLI?

The .NET CLI (dotnet command) is the cross-platform tool for creating, building, running, testing, and publishing .NET applications. Everything Visual Studio does under the hood, the CLI does explicitly.

```
# Core commands:
dotnet new      # create new projects and files
dotnet build    # compile the project
dotnet run      # build and run
dotnet test     # run tests
dotnet publish  # publish for deployment
dotnet restore  # restore NuGet packages
dotnet add      # add packages or project references
dotnet list     # list references or packages
```

Creating projects with dotnet new

```
# List all available templates:
dotnet new list

# Common templates:
dotnet new webapi    # ASP.NET Core Web API
dotnet new mvc       # ASP.NET Core MVC
dotnet new console   # Console application
dotnet new classlib  # Class library
dotnet new xunit     # xUnit test project
dotnet new sln       # Solution file

# Create with name, location, framework:
dotnet new webapi -n MyApp.Api -o src/MyApp.Api
dotnet new webapi -n MyApp.Api --framework net9.0
dotnet new webapi -n MyApp.Api --no-https
```

Solution and project structure (Clean Architecture)

```
MyApp.sln
■■■ src/
■ ■■■ MyApp.Api/          <- ASP.NET Core API (presentation)
```

```

■ ■ ■■ MyApp.Api.csproj
■ ■■ MyApp.Application/ <- Business logic, interfaces
■ ■ ■■ MyApp.Application.csproj
■ ■■ MyApp.Domain/ <- Entities, value objects
■ ■ ■■ MyApp.Domain.csproj
■ ■■ MyApp.Infrastructure/<- EF Core, repos, external svcs
■ ■■ MyApp.Infrastructure.csproj
■■■ tests/
    ■■ MyApp.UnitTests/
    ■■ MyApp.IntegrationTests/

```

Creating a solution with multiple projects

```

dotnet new sln -n MyApp

dotnet new webapi -n MyApp.Api -o src/MyApp.Api
dotnet new classlib -n MyApp.Application\
    -o src/MyApp.Application
dotnet new classlib -n MyApp.Domain -o src/MyApp.Domain
dotnet new classlib -n MyApp.Infrastructure\
    -o src/MyApp.Infrastructure
dotnet new xunit -n MyApp.UnitTests\
    -o tests/MyApp.UnitTests

dotnet sln add src/MyApp.Api/MyApp.Api.csproj
dotnet sln add src/MyApp.Application/MyApp.Application.csproj
dotnet sln add src/MyApp.Domain/MyApp.Domain.csproj
dotnet sln add src/MyApp.Infrastructure/MyApp.Infrastructure.csproj
dotnet sln add tests/MyApp.UnitTests/MyApp.UnitTests.csproj

# Add project references (dependency direction):
dotnet add src/MyApp.Api/MyApp.Api.csproj reference\
    src/MyApp.Application/MyApp.Application.csproj
dotnet add src/MyApp.Application/MyApp.Application.csproj\
    reference src/MyApp.Domain/MyApp.Domain.csproj

```

The .csproj file

```

<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <ImplicitUsings>enable</ImplicitUsings>
    <RootNamespace>MyApp.Api</RootNamespace>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference
      Include="Microsoft.EntityFrameworkCore.SqlServer"
      Version="9.0.*" />
  </ItemGroup>
  <ItemGroup>
    <ProjectReference
      Include="..\MyApp.Application\MyApp.Application.csproj"
    />
  </ItemGroup>
</Project>

```

```
</ItemGroup>
</Project>
```

Managing NuGet packages

```
# Add a package:
dotnet add package Microsoft.EntityFrameworkCore.SqlServer

# Add to a specific project:
dotnet add src/MyApp.Infrastructure/MyApp.Infrastructure.csproj\
    package Microsoft.EntityFrameworkCore.SqlServer

# Remove a package:
dotnet remove package Microsoft.EntityFrameworkCore.SqlServer

# List outdated packages:
dotnet list package --outdated

# Restore all packages (after cloning):
dotnet restore
```

Build, run, test, publish

```
# Build:
dotnet build
dotnet build --configuration Release

# Run:
dotnet run --project src/MyApp.Api/MyApp.Api.csproj
dotnet run --project src/MyApp.Api --environment Development

# Test:
dotnet test
dotnet test --filter "Category=Unit"
dotnet test --verbosity normal

# Publish:
dotnet publish -c Release -o ./publish

# Self-contained (no .NET runtime needed on server):
dotnet publish -c Release -r linux-x64 --self-contained true
```

Global tools and EF Core migrations

```
# Install EF Core CLI tool:
dotnet tool install --global dotnet-ef

# Create a migration:
dotnet ef migrations add InitialCreate\
    --project src/MyApp.Infrastructure\
    --startup-project src/MyApp.Api

# Apply migrations:
dotnet ef database update\
    --project src/MyApp.Infrastructure\
```

```
--startup-project src/MyApp.Api
```

Key Takeaways for 12.1

- dotnet CLI is the foundation -- create, build, run, test, publish
- Solution (.sln) = container for multiple projects
- Project (.csproj) = one compilable unit
- Clean Architecture: separate projects for Api, Application, Domain, Infra, Tests
- dotnet add package adds NuGet; dotnet add reference adds project reference
- launchSettings.json is local-only -- sets env vars for development

Command	Purpose
-----+-----	
dotnet new webapi -n Name	Create a Web API project
dotnet sln add <proj>	Add project to solution
dotnet add package <pkg>	Add NuGet package
dotnet build	Compile
dotnet run --project <p>	Run a project
dotnet test	Run tests
dotnet publish -c Release	Build for deployment
dotnet ef migrations add	Create EF migration

Memory aid: The dotnet CLI is the Swiss Army knife of .NET. new creates, build compiles, run executes, test validates, publish packages. You can do everything from the terminal without an IDE.

Debugging Tips

Symptom	Cause	Fix
dotnet run fails with project not found	Running from wrong directory or missing --project flag	cd to solution root; use --project src/MyApp.Api/MyApp.Api.csproj
NuGet restore fails on CI	NuGet source not configured or private feed missing credentials	Add nuget.config to repo root with source URLs
dotnet ef not found after install	Global tools path not in PATH environment variable	Add ~/.dotnet/tools to PATH; restart terminal
Build succeeds locally but fails on CI	Different .NET SDK version or missing global.json	Add global.json pinning the SDK version to the repo root

12.6. Hosting Model (Generic Host, WebApplicationBuilder)

Builds on: 12.1 (CLI -- run with dotnet run), 9.5 (DI -- host is where DI container is built). Taught before 12.2/12.3 because Program.cs is where DI registration happens.

What is the hosting model?

The .NET Generic Host is the runtime container that starts and stops your app, manages the DI container, manages configuration, manages logging, and runs hosted services. From .NET 6 onwards, everything is in Program.cs using the minimal hosting API.

[illegible]

```
var builder = WebApplication.CreateBuilder(args);
// Phase 1: BUILDER
// builder.Services           -- register services
// builder.Configuration      -- read configuration
// builder.Logging            -- configure logging
// builder.Environment        -- current environment
// builder.Host               -- configure Generic Host
// builder.WebHost            -- configure Kestrel

var app = builder.Build();
// Phase 2: APP (pipeline)
// app.Use*()                -- add middleware
// app.Map*()                 -- add endpoints
// app.Run()                  -- start the server
```

```
// After Build() you cannot register new services
```

Generic Host for background workers

```
// Worker service -- no HTTP:
var host = Host.CreateDefaultBuilder(args)
    .ConfigureServices((context, services) =>
    {
        services.AddHostedService<OrderProcessingWorker>();
        services.AddScoped<IOrderRepository,
            SqlOrderRepository>();
        services.AddDbContext<AppDbContext>(options =>
            options.UseSqlServer(
                context.Configuration
                    .GetConnectionString("Default")));
    })
    .Build();

await host.RunAsync();

// The worker:
public class OrderProcessingWorker : BackgroundService
{
    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await ProcessPendingOrdersAsync(stoppingToken);
            await Task.Delay(
                TimeSpan.FromSeconds(30), stoppingToken);
        }
    }
}
```

Middleware pipeline -- order matters

```
// Register in this order:
app.UseExceptionHandler("/error"); // 1. catch all exceptions
app.UseHttpsRedirection();          // 2. redirect to HTTPS
app.UseStaticFiles();               // 3. serve static files
app.UseRouting();                   // 4. match routes
app.UseCors();                      // 5. CORS headers
app.UseAuthentication();            // 6. identify the user
app.UseAuthorization();             // 7. check permissions
app.MapControllers();               // 8. dispatch to controller

// Rule: UseAuthentication BEFORE UseAuthorization
// Rule: Exception handler FIRST to catch all errors
```

Key Takeaways for 12.6

- Generic Host manages DI, configuration, logging, and app lifetime
- WebApplicationBuilder: builder phase (register) -> app phase (pipeline)

- After `builder.Build()` DI container is sealed -- no more registrations
- Middleware order matters -- auth before authz, exception handler first
- `BackgroundService` uses Generic Host without the web stack

Memory aid: `WebApplicationBuilder` is a restaurant being set up before opening. You arrange tables (register services) before the doors open. `builder.Build()` opens the doors. After that you cannot rearrange furniture.

Debugging Tips

Symptom	Cause	Fix
Service not resolved after registration	Registered after <code>builder.Build()</code> -- too late	Move all registrations before <code>var app = builder.Build()</code>
Middleware not executing for some routes	Middleware registered after <code>MapControllers()</code>	Register middleware before <code>MapControllers()</code>
<code>app.UseAuthentication</code> has no effect	Registered after <code>UseAuthorization</code> -- wrong order	Always <code>UseAuthentication</code> before <code>UseAuthorization</code>
<code>BackgroundService</code> cannot resolve Scoped service	<code>BackgroundService</code> is Singleton -- captive dependency	Inject <code>IServiceScopeFactory</code> ; create scope per work unit

12.2. Dependency Injection Container (IServiceCollection)

Builds on: 9.5 (manual DI -- you already understand DI conceptually; this adds the container mechanics), 12.6 (hosting -- services registered on `builder.Services`).

IServiceCollection -- the registration API

```
// Three lifetime registration methods:
builder.Services.AddSingleton<IService, Implementation>();
builder.Services.AddScoped<IService, Implementation>();
builder.Services.AddTransient<IService, Implementation>();

// By interface -> implementation (most common):
builder.Services
    .AddScoped<IOrderRepository, SqlOrderRepository>();

// By concrete type only (no interface):
builder.Services.AddScoped<OrderService>();

// By factory (when creation needs logic):
builder.Services.AddScoped<IPaymentProcessor>(provider =>
{
    var config =
        provider.GetRequiredService<IConfiguration>();
    return config["PaymentProvider"] == "Stripe"
        ? new StripeProcessor(config["Stripe:ApiKey"]!)
        : new PaypalProcessor(config["PayPal:ClientId"]!);
});

// Pre-existing instance as singleton:
```

```

var settings = new SmtplibSettings { Host = "smtp.zoho.com" };
builder.Services.AddSingleton(settings);

// Open generic registration:
builder.Services.AddScoped(
    typeof(IRepository<>), typeof(EfRepository<>));

```

Built-in extension methods

```

// Controllers + model binding + validation:
builder.Services.AddControllers();

// EF Core DbContext:
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(connectionString));

// Named HttpClient:
builder.Services.AddHttpClient("PaymentApi", client =>
{
    client.BaseAddress =
        new Uri("https://api.payment.com");
    client.DefaultRequestHeaders
        .Add("Accept", "application/json");
});

// Typed HttpClient:
builder.Services.AddHttpClient<PaymentApiClient>(c =>
    c.BaseAddress = new Uri("https://api.payment.com"));

// Memory cache:
builder.Services.AddMemoryCache();

// CORS:
builder.Services.AddCors(options =>
    options.AddDefaultPolicy(policy =>
        policy.WithOrigins("https://myapp.com")
            .AllowAnyMethod()
            .AllowAnyHeader()));

```

Registering multiple implementations

```

// Register multiple implementations of same interface:
builder.Services
    .AddScoped<INotificationSender, EmailSender>();
builder.Services
    .AddScoped<INotificationSender, SmsSender>();
builder.Services
    .AddScoped<INotificationSender, PushSender>();

// Inject all via IEnumerable<T>:
public class NotificationService
{
    private readonly IEnumerable<INotificationSender>
        _senders;

```

```

public NotificationService(
    IEnumerable<INotificationSender> senders)
    => _senders = senders;

public async Task SendAllAsync(string message)
{
    foreach (var sender in _senders)
        await sender.SendAsync(message);
}
}

// Note: injecting single INotificationSender gets
// the LAST registered implementation (PushSender)

```

Keyed services (.NET 8+)

```

// Register with a key:
builder.Services.AddKeyedScoped<IPaymentProcessor,
    StripeProcessor>("stripe");
builder.Services.AddKeyedScoped<IPaymentProcessor,
    PaypalProcessor>("paypal");

// Inject by key:
public class CheckoutService
{
    private readonly IPaymentProcessor _processor;

    public CheckoutService(
        [FromKeyedServices("stripe")]
        IPaymentProcessor processor)
        => _processor = processor;
}

```

Key Takeaways for 12.2

- IServiceCollection is the list of service registrations
- Factory registration: AddScoped<IFoo>(provider => ...) for creation logic
- Open generic: AddScoped(typeof(IRepo<>), typeof(EfRepo<>)) covers all T
- Multiple registrations: inject IEnumerable<T> to get all
- Single T injection: gets the last registered implementation
- Keyed services (.NET 8+): inject specific impl by name with FromKeyedServices

Memory aid: IServiceCollection is the staffing roster before the office opens. You tell the agency what roles you need and who fills them. builder.Build() opens the office. After that you cannot hire new staff mid-day.

Debugging Tips

Symptom	Cause	Fix
InvalidOperationException: No service for type IXxx	Interface not registered in DI	Add builder.Services.AddScoped<IXxx, ConcreteXxx>()


```

public class OrderCache // registered Singleton
{
    // IOrderRepository is Scoped!
    private readonly IOrderRepository _repo;

    // Scoped repo captured at first construction.
    // Never released -- same repo instance forever.
    // Subsequent requests use a STALE DbContext.
    public OrderCache(IOrderRepository repo)
        => _repo = repo;
}

// ASP.NET Core detects this at startup in Development:
// InvalidOperationException:
// Cannot consume scoped service 'IOrderRepository'
// from singleton 'OrderCache'.

// Safe dependency rules:
// Singleton can depend on: Singleton
// Scoped can depend on: Singleton + Scoped
// Transient can depend on: All three

```

IServiceScopeFactory -- Singleton accessing Scoped

```

public class OrderProcessingWorker : BackgroundService
{
    private readonly IServiceScopeFactory _scopeFactory;

    public OrderProcessingWorker(
        IServiceScopeFactory scopeFactory)
        => _scopeFactory = scopeFactory;

    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            // Create a scope per unit of work:
            using var scope =
                _scopeFactory.CreateScope();
            var repo = scope.ServiceProvider
                .GetRequiredService<IOrderRepository>();

            await ProcessPendingOrdersAsync(
                repo, stoppingToken);
            await Task.Delay(
                TimeSpan.FromSeconds(30), stoppingToken);
            // Scope disposed here -- DbContext released
        }
    }
}

```

Choosing the right lifetime

Question	Lifetime
----------	----------

	+
Stateless utility, no shared data?	Transient
Holds DbContext or connection?	Scoped
Expensive to create, thread-safe?	Singleton
Per-request consistent state needed?	Scoped
App-wide shared cache or config?	Singleton

Key Takeaways for 12.3

- Transient -- new each injection -- stateless helpers, validators
- Scoped -- one per HTTP request -- DbContext, unit of work
- Singleton -- one forever -- caches, config, thread-safe services
- Captive dependency: Singleton holding Scoped = bug, stale data
- IServiceScopeFactory -- create explicit scopes inside BackgroundService
- ASP.NET Core validates captive deps at startup in Development mode

Memory aid: Transient = paper cup (use once, throw away). Scoped = a glass washed after each meal service. Singleton = the restaurant stove -- one for the entire lifetime, shared by all.

Debugging Tips

Symptom	Cause	Fix
Cannot consume scoped service from singleton	Captive dependency -- Singleton holding a Scoped service	Change Singleton to Scoped, or use IServiceScopeFactory
DbContext shows stale data across requests	DbContext registered as Singleton instead of Scoped	Use AddDbContext which registers as Scoped by default
Transient service holds state between calls	Service captured in a Singleton field	Ensure the Singleton only holds Singleton dependencies
Scope validation not catching bug	Running in Production mode -- validation only in Development	Always test in Development before promoting to Staging

12.4. Configuration System

Builds on: 12.6 (builder.Configuration set up by CreateBuilder), 12.2 (DI -- options registered and injected via DI), 9.5 (constructor injection -- options injected through constructors).

How configuration loading works

```
# Priority (highest wins):
5. Environment variables          <- highest priority
4. Command-line arguments
3. User secrets (Development only)
2. appsettings.{Environment}.json
1. appsettings.json              <- lowest priority

# WebApplication.CreateBuilder sets all this up automatically
```

appsettings.json -- the base configuration

```

{
  "ConnectionStrings": {
    "DefaultConnection":
      "Server=prod-db;Database=Orders;..."
  },
  "SmtpSettings": {
    "Host": "smtp.zoho.com",
    "Port": 587,
    "FromAddress": "orders@myapp.com"
  },
  "FeatureFlags": {
    "EnableOrderExport": true,
    "MaxOrdersPerPage": 50
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}

```

The Options pattern -- strongly-typed configuration

```

// 1. Define a settings class:
public class SmtpSettings
{
    public const string SectionName = "SmtpSettings";

    public string Host { get; set; } = string.Empty;
    public int Port { get; set; }
    public string FromAddress { get; set; } = string.Empty;
    public string Username { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}

// 2. Register in Program.cs:
builder.Services.Configure<SmtpSettings>(
    builder.Configuration
        .GetSection(SmtpSettings.SectionName));

// 3. Inject IOptions<T> in your service:
public class SmtpEmailService : IEmailService
{
    private readonly SmtpSettings _settings;

    public SmtpEmailService(IOptions<SmtpSettings> options)
        => _settings = options.Value;

    public async Task SendAsync(
        string to, string subject, string body)
    {
        using var client = new SmtpClient();
        await client.ConnectAsync(
            _settings.Host, _settings.Port);
    }
}

```

```

    }
}

```

IOptions vs IOptionsSnapshot vs IOptionsMonitor

```

// IOptions<T> -- fixed at startup, Singleton lifetime:
public class MyService(IOptions<SmtpSettings> options)
{
    private readonly SmtpSettings _s = options.Value;
}
// Use for: Singleton services, static config

// IOptionsSnapshot<T> -- re-read per request, Scoped:
public class MyService(IOptionsSnapshot<SmtpSettings> opts)
{
    private readonly SmtpSettings _s = opts.Value;
}
// Use for: Scoped services, config that may change

// IOptionsMonitor<T> -- live updates, any lifetime:
public class MyService(IOptionsMonitor<SmtpSettings> mon)
{
    // CurrentValue always reads the latest value:
    public string Host => mon.CurrentValue.Host;
}
// Use for: Singletons that react to config changes

```

User secrets -- for local development

```

# Initialize user secrets:
dotnet user-secrets init --project src/MyApp.Api

# Set secrets:
dotnet user-secrets set\
    "ConnectionStrings:DefaultConnection"\
    "Server=localhost;Database=MyAppDev;..." \
    --project src/MyApp.Api

dotnet user-secrets set "SmtpSettings:Password"\
    "mypassword123" --project src/MyApp.Api

# List all secrets:
dotnet user-secrets list --project src/MyApp.Api

# Stored outside the project directory:
# Win: %APPDATA%\Microsoft\UserSecrets\<guid>\secrets.json
# Linux: ~/.microsoft/usersecrets/<guid>/secrets.json

```

Environment variables -- for production

```

# Use __ (double underscore) as section separator:

# Equivalent to ConnectionStrings.DefaultConnection:
export ConnectionStrings__DefaultConnection="Server=..."

# In Docker:

```

```
docker run\
-e "ConnectionStrings__DefaultConnection=Server=db;"\
myapp
```

```
# In Kubernetes (deployment YAML):
env:
- name: ConnectionStrings__DefaultConnection
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: connection-string
```

Validating options at startup

```
public class SntpSettings
{
    [Required]
    public string Host { get; set; } = string.Empty;

    [Range(1, 65535)]
    public int Port { get; set; }

    [Required, EmailAddress]
    public string FromAddress { get; set; } = string.Empty;
}

// Register with validation:
builder.Services
    .AddOptions<SntpSettings>()
    .BindConfiguration(SntpSettings.SectionName)
    .ValidateDataAnnotations()
    .ValidateOnStart(); // throws at startup if invalid
```

Key Takeaways for 12.4

- Config loads from multiple sources -- env vars override appsettings.json
- Options pattern -- typed settings class + `Configure<T>` + `IOptions<T>` injection
- `IOptions<T>` fixed at startup; `IOptionsSnapshot<T>` re-read per request
- User secrets -- local dev secrets outside source control
- Env vars use `__` as section separator: `SntpSettings__Password`
- `ValidateOnStart()` -- fail fast with clear error if config is missing

Memory aid: appsettings.json is the default recipe. User secrets are your personal sticky note. Environment variables are the executive chef overriding everything. The Options pattern puts the final recipe into a typed card the cooks can read.

Debugging Tips

Symptom	Cause	Fix
<code>IOptions.Value</code> returns default values	Section name mismatch or <code>BindConfiguration</code> not called	Check <code>GetSection</code> name matches JSON key exactly (case-sensitive)

Symptom	Cause	Fix
User secrets not loading	Not in Development environment or secrets not initialized	Ensure ASPNETCORE_ENVIRONMENT=Development; run dotnet user-secrets init
Environment variable not overriding appsettings	Wrong separator -- using : instead of __ in env var name	Use __ (double underscore): SmtpSettings__Host not SmtpSettings:Host
ValidateOnStart not catching missing config	ValidateDataAnnotations not called before ValidateOnStart	Chain: .ValidateDataAnnotations().ValidateOnStart()

12.5. Logging (ILogger, Providers)

Builds on: 12.6 (hosting -- logging configured on builder), 12.2 (DI -- ILogger<T> injected), 12.7 (environments -- log levels change per environment).

ILogger<T> -- the standard logging abstraction

```
public class OrderService
{
    private readonly ILogger<OrderService> _logger;
    private readonly IOrderRepository _repo;

    public OrderService(
        ILogger<OrderService> logger,
        IOrderRepository repo)
    {
        _logger = logger;
        _repo = repo;
    }

    public async Task<Order?> GetByIdAsync(
        int id, CancellationToken ct)
    {
        _logger.LogInformation(
            "Fetching order {OrderId}", id);

        var order = await _repo.GetByIdAsync(id, ct);

        if (order is null)
            _logger.LogWarning(
                "Order {OrderId} not found", id);

        return order;
    }
}
```

Log levels -- severity order

```
// From lowest to highest severity:
_logger.LogTrace("Entering GetByIdAsync id={Id}", id);
// Very detailed -- method entry/exit, variable values
```

```

_logger.LogDebug("Cache miss for order {Id}", id);
// Diagnostic info useful during development

_logger.LogInformation(
    "Order {Id} created by {User}", id, user);
// Normal operational events -- requests, state changes

_logger.LogWarning(
    "Order {Id} not found -- returning null", id);
// Unexpected but handled

_logger.LogError(ex,
    "Failed to save order {Id}", id);
// Error that prevented an operation

_logger.LogCritical(ex,
    "Database connection lost");
// System-level failure -- app may need restart

```

Structured logging -- use message templates

```

// WRONG -- string interpolation loses structure:
_logger.LogInformation(
    $"Order {order.Id} created for {customer.Name}");
// Logged as plain text -- cannot query by OrderId

// CORRECT -- named placeholders:
_logger.LogInformation(
    "Order {OrderId} created for {CustomerName}",
    order.Id, customer.Name);
// Logged with structured properties:
// OrderId=42, CustomerName="Sathish"
// Queryable in Seq/Elasticsearch/App Insights

```

High-performance logging with LoggerMessage

```

// Define once (static -- no allocation on hot path):
public static partial class LogMessages
{
    [LoggerMessage(
        Level = LogLevel.Information,
        Message = "Order {OrderId} processed in {Ms}ms")]
    public static partial void OrderProcessed(
        ILogger logger, int orderId, long ms);

    [LoggerMessage(
        Level = LogLevel.Error,
        Message = "Failed to process order {OrderId}")]
    public static partial void OrderFailed(
        ILogger logger, Exception ex, int orderId);
}

// Usage:
LogMessages.OrderProcessed(
    _logger, order.Id, stopwatch.ElapsedMilliseconds);

```

```
LogMessages.OrderFailed(_logger, ex, order.Id);
```

Configure log levels in appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore
        .Database.Command": "Information",
      "MyApp.Infrastructure": "Debug"
    }
  }
}
```

Adding Serilog

```
# dotnet add package Serilog.AspNetCore
# dotnet add package Serilog.Sinks.File

// Program.cs:
builder.Host.UseSerilog(
    (context, services, configuration) =>
        configuration
            .ReadFrom.Configuration(context.Configuration)
            .ReadFrom.Services(services)
            .Enrich.FromLogContext()
            .WriteTo.Console()
            .WriteTo.File(
                "logs/myapp-.txt",
                rollingInterval: RollingInterval.Day));
```

Key Takeaways for 12.5

- `ILogger<T>` -- inject and use this; never reference a specific lib in code
- Six log levels: Trace -> Debug -> Information -> Warning -> Error -> Critical
- Structured logging -- use `{PlaceholderName}` templates, never string interpolation
- Configure log levels per namespace in `appsettings.json`
- Serilog -- popular third-party library; adds file/Seq/cloud sinks
- `LoggerMessage` source generators -- high-performance for hot paths

Memory aid: `ILogger` is like a dictaphone. You speak into it (log). The listener (provider) decides where the recording goes -- console, file, Seq. Your code never knows or cares where it ends up.

Debugging Tips

Symptom	Cause	Fix
Logs not appearing despite logging calls	Log level filtered out -- Default set too high	Lower <code>LogLevel.Default</code> to Debug in <code>appsettings.Development.json</code>
EF Core SQL queries not showing in logs	EF Command log category not set to Information	Add <code>Microsoft.EntityFrameworkCore.Database.Command: Information</code>

Symptom	Cause	Fix
Structured properties not appearing in Seq/Elastic	Using string interpolation instead of message template	Replace \$"Order {id}" with "Order {OrderId}", id
Serilog not writing to file	Output path does not exist or insufficient permissions	Create logs/ directory; check write permissions

12.7. Environments (Development/Staging/Production)

Builds on: 12.6 (hosting -- app.Environment set by host), 12.4 (configuration -- appsettings.{Environment}.json).

What are environments?

The ASPNETCORE_ENVIRONMENT variable tells your app which environment it is running in. Three standard values: Development (local machine), Staging (pre-production), Production (live, customer-facing).

```
// Conditional behaviour based on environment:
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
    app.UseDeveloperExceptionPage();
}
else
{
    // Safe error page in non-dev:
    app.UseExceptionHandler("/error");
    app.UseHsts();
}

// Custom environments:
if (app.Environment.IsEnvironment("Staging"))
{
    // staging-specific behaviour
}
```

Environment-specific configuration files

```
appsettings.json          <- base, all environments
appsettings.Development.json <- overrides for Development
appsettings.Staging.json   <- overrides for Staging
appsettings.Production.json <- overrides for Production

// Deep-merges on top of the base:

// appsettings.json (base):
{
    "Logging": { "LogLevel": { "Default": "Information" } },
    "FeatureFlags": { "EnableOrderExport": false }
}
```

```
// appsettings.Development.json (overrides):
{
  "Logging": { "LogLevel": { "Default": "Debug" } },
  "FeatureFlags": { "EnableOrderExport": true }
}
```

Setting the environment

```
# In launchSettings.json (local dev):
"environmentVariables": {
  "ASPNETCORE_ENVIRONMENT": "Development"
}

# Terminal / CI:
export ASPNETCORE_ENVIRONMENT=Staging
dotnet run

# In Docker:
docker run -e ASPNETCORE_ENVIRONMENT=Production myapp

# Azure App Service:
# Application Settings -> ASPNETCORE_ENVIRONMENT = Production
```

What changes per environment

Development:

- Swagger UI enabled
- Detailed exception pages
- DI scope validation at startup
- Debug logging
- User secrets loaded

Staging:

- No detailed exceptions
- Warning+ logging
- Real database (staging copy)

Production:

- No Swagger
- No detailed exceptions
- Error+ logging
- Real database
- HSTS and HTTPS enforced

IWebHostEnvironment -- injecting environment info

```
public class DiagnosticsController : ControllerBase
{
  private readonly IWebHostEnvironment _env;

  public DiagnosticsController(IWebHostEnvironment env)
    => _env = env;

  [HttpGet("health")]
  public IActionResult Health()
```

```

{
    var info = new
    {
        Environment = _env.EnvironmentName,
        // Only expose details in non-production:
        Details = _env.IsProduction() ? null : new
        {
            MachineName = Environment.MachineName,
            ProcessId    = Environment.ProcessId
        }
    };
    return Ok(info);
}
}

```

Key Takeaways for 12.7

- ASPNETCORE_ENVIRONMENT controls active environment
- appsettings.{Environment}.json deep-merges over appsettings.json
- IsDevelopment() -- Swagger, detailed errors, debug logging only in dev
- DI scope validation runs automatically in Development
- Inject IWebHostEnvironment to access environment info anywhere
- Production: no detailed exceptions, no Swagger, minimal logging, HTTPS enforced

Memory aid: Environments are like a restaurant's service modes. Development is the test kitchen -- everything visible, errors displayed, staff learning. Production is the real service -- clean presentation, errors hidden from guests, everything polished.

Debugging Tips

Symptom	Cause	Fix
Swagger not showing in Development	ASPNETCORE_ENVIRONMENT not set to Development	Check launchSettings.json; set env var explicitly
DI scope validation not catching bugs	Running in Production -- validation only in Development	Always test in Development before promoting to Staging/Production
appsettings.Development.json not loaded	ASPNETCORE_ENVIRONMENT not set to Development	Ensure environment variable is set correctly before running
Environment-specific file overriding all settings	Environment file has a top-level key overriding entire section	Use deep merge -- only specify the keys you want to override

Section 12 -- Complete Key Takeaways

Topic	Core concept	Key API
12.1 CLI	Build, run, publish	dotnet new/run
12.2 DI Container	Register on IServiceColl	builder.Services
12.3 Lifetimes	Transient/Scoped/Single	Add*/Singleton
12.4 Configuration	Options pattern	IOptions<T>
12.5 Logging	Structured via ILogger	LogInformation

12.6 Hosting	Builder+pipeline	WebApplicationBuilder
12.7 Environments	Behaviour per env	IsDevelopment()

Section 12 complete. All 7 roadmap topics (12.1 through 12.7) covered: .NET CLI and project structure, DI container, service lifetimes, configuration system, logging, hosting model, and environments. Say next to begin Section 13: ASP.NET Core Web API Fundamentals.